

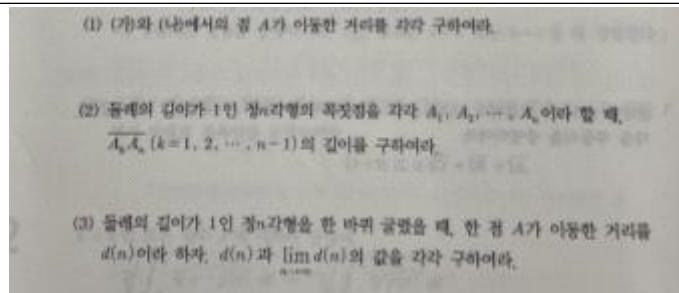
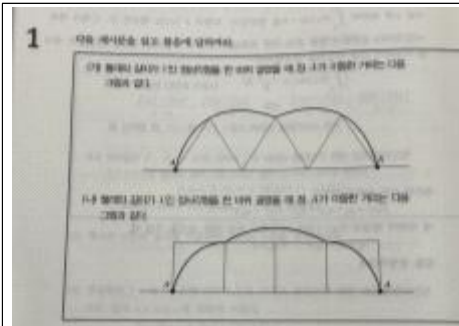


수학 시뮬레이터 제작 보고서

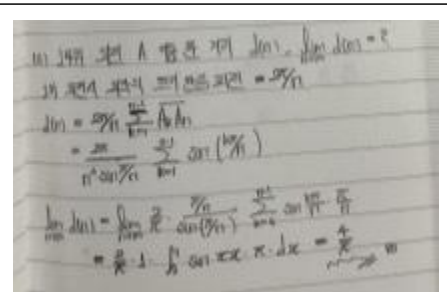
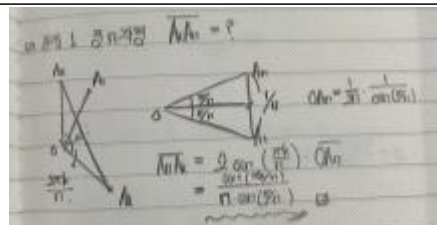
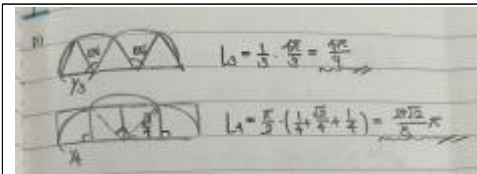
수행평가

[첫 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 첫 번째 문제 (캡처해서 붙여넣기 해도 됨)



2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)



3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

이 문제는 단순히 정n각형의 성질을 계산하는 문제가 아니라, 도형이 실제로 굴러가는 과정 속에서 점 A의 움직임이 어떻게 변하는지를 추적해야 하는 동적 기하 문제이다. 특히 n의 값에 따라 회전각과 점 A가 그리는 원호의 반지름이 계속 달라지기 때문에, 정적인 그림이나 식만으로는 전체 구조를 직관적으로 이해하기 어렵다고 느꼈다. 그래서 다음과 같은 이유로 시뮬레이션이 필요하다고 생각했다.

첫 번째로 정n각형의 굴러가는 과정을 시각적으로 이해할 필요가 있었다. (문제 파악)

문제를 처음 보았을 때는 점 A의 자취가 왜 여러 개의 원호로 이루어지는지 직관적으로 떠오르지 않았다. 특히 정삼각형, 정사각형, 정10각형처럼 n이 달라질 때마다 회전하는 방식과 자취의 형태가 무엇이 달라지는지 머릿속으로 상상하기 어려웠을 것 같다. 시뮬레이터를 통해 도형이 실제로 한 번씩 회전하며 이동하는 모습을 직접 확인해 보면, 점 A의 자취가 각 단계의 회전 운동의 합이라는 사실을 훨씬 쉽게 이해할 수 있을 것 같았다.

두 번째로 조건 (2), (3)의 식 구조를 검토하고 일반화 과정을 확인하기 위함이다. (검토)

이 문제에서는 각 단계의 회전각이 항상 외각으로 일정하고, 반지름만 달라진다는 사실을 발견하는 것이 중요하다. 하지만 식만 보면 왜 이동 거리가 이런 꼴로 정리되는지 바로 와닿지 않는다. 시뮬레이터에서 각 단계의 회전 중심과 원호를 직접 표시해 보며, “중심각 × 반지름”이 반복되는 구조를 시각적으로 확인할 수 있었고, 내가 세운 식이

실제 움직임과 일치하는지 검토하는 데 도움을 받을 수 있을 것 같다.

세 번째로, 마지막 극한 계산을 위한 기하학적 인사이트를 얻기 위함이다. (폴이에 대한 힌트)

n 이 커질수록 정 n 각형이 점점 원에 가까워지고, 점 A의 자취도 매우 부드러운 사이클로이드 곡선 형태로 변하게 된다. 이 모습을 시뮬레이터로 직접 관찰해 보니, 각 단계의 합이 단순한 유한합이 아니라 잘게 나누어진 넓이 혹은 곡선의 합처럼 보였고, 이것이 리만합과 정적분으로 연결된다는 사실을 직관적으로 이해할 수 있었다. 즉, 단순히 계산을 위한 도구가 아니라, 왜 극한이 적분 형태로 바뀌는지에 대한 핵심 아이디어를 제공해줄 수 있었다.

4. 사용한 AI 플랫폼을 모두 적으세요.

ChatGPT , Gemini

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

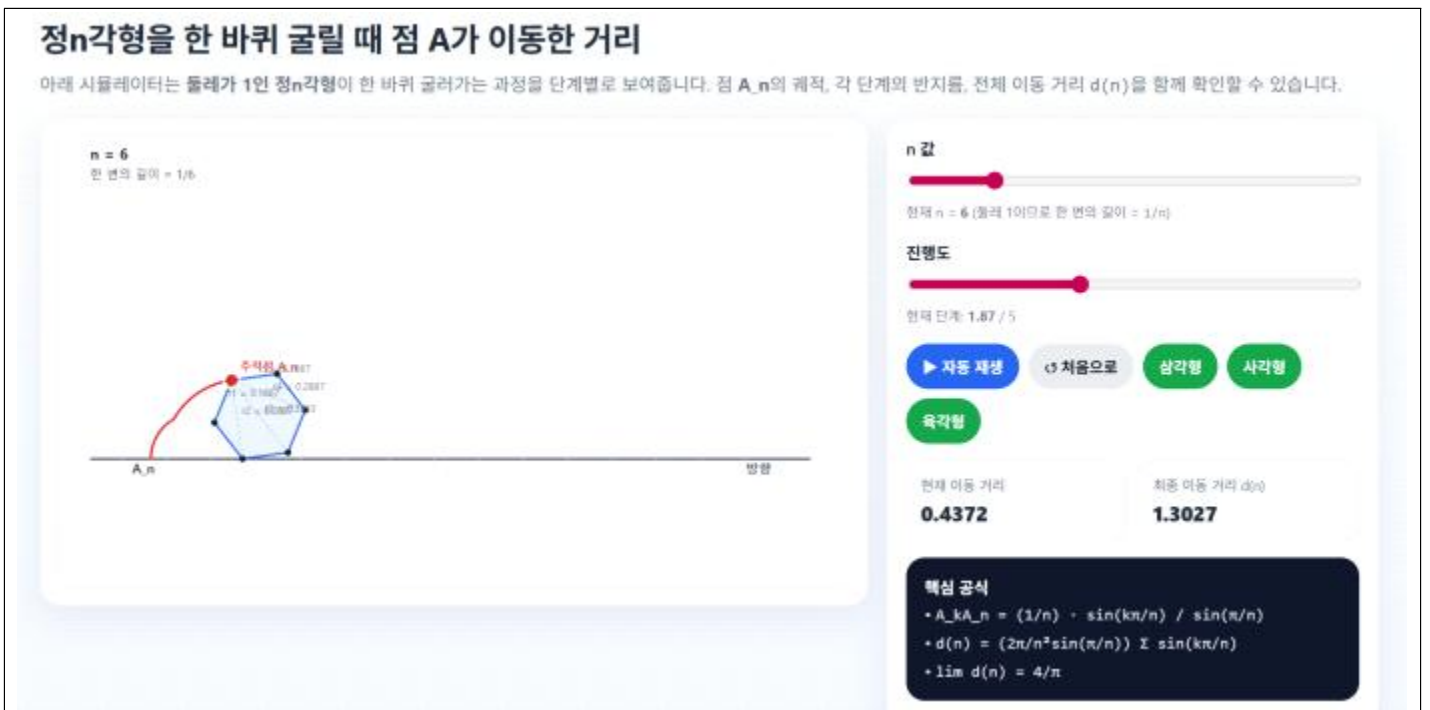
문제 사진과 내가 직접 푼 글과 선생님의 예시 파일을 넣으며....

먼저 너가 직접 한번 문제를 풀어보고, 내가 푼 풀이와 비교하며 먼저 문제의 핵심과 답을 파악 해줘.

정 n 각형이 한 바퀴 굴러갈 때 점 A의 이동거리를 보여주는 HTML 시뮬레이터를 만들어줘.

정삼각형, 정사각형, 원 아니면 정100각형을 버튼으로 바로 바꿀 수 있게 하고, 슬라이더로 n 과 속도를 조절할 수 있게 해줘. 점 A의 궤적이 화면에 표시되도록 하고, 현재 $d(n)$ 값도 함께 보여줘.

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

최초 시뮬레이터는 정 n 각형에 따라서 변의 개수가 증가할때 변화하는 개형을 쉽게 파악할 수 있어 시뮬레이션의 의도에는 적합했다. 하지만 시뮬레이션을 실행하는 과정에서 보기 불편한 부분이 몇 부분 존재하였다.

첫 번째로, 도형이 회전할 때 3각형, 4각형이 회전할 때는 적절한 속도로 회전하였지만, 변의 개수가 많아질수록

각속도의 값이 줄어 매우 미세하고 느리게 움직이는 문제점이 있었다. 리만합을 극한으로 보내 원의 자취 즉 사이클로이드의 운동을 관찰하기 위해 속도를 증가 조절할 필요가 있었다.

두 번째로, n 의 값을 원 즉 무한대까지 요청했지만, 코딩의 한계로 20각형까지만 구현되어 원의 운동을 보기 위해서는 가능한 n 의 값의 범위를 늘릴 필요가 있었다.

다음으로 숫자가 소수로 표현되어 무리수 값(파이와 루트)들이 잘 나타나지 않아 수학적으로 자취의 길이를 파악하기 어려웠다. 문제를 해결하기 위하여 소수점을 없애고 무리수 형태로 답을 표현해야 한다.

마지막으로, 문제에서 무엇을 물어보는지 한눈에 들어오지 않는 디자인이 문제였다. 마지막에 핵심 공식으로 1,2,3번의 핵심을 정리하긴 했지만, 그 값이 왜 그런 값을 가지는지 중간 과정이 생략되어 있고, 한 곳에 밀집해 있어 아무런 의미가 없는 정리라는 느낌이 들어 디자인을 수정하였다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

(2) 아니야 너무 화면이 세로로 길어. 위치를 잘 활용해 봐.

(3) 그리고 초록색 버튼은 문제에서 물어본 3/4각형 그리고 100각형(원)으로 해줘. 재생 속도 혹시 범위를 더 늘려줘. 그리고 각 문제를 어떻게 풀었는지 짧은 식과 한국어로 설명도 넣어줘. 검은색 박스에 다 있으니까 너무 칙칙해 보여

(4) 아니 형태는 잘 바꿨는데 갑자기 작동이 안 함. 수정 해줘 오류 사진은 이래

(5) 내가 파일을 2개 보낼건데 simulator파일은 운동은 완벽한데 디자인이 살짝 별로 이고 final 파일은 운동이 지금 잘못되고 있는데 디자인은 그렇게 해야해 너가 둘을 잘 융합해서 완벽한 운동과 디자인의 시뮬레이션을 만들어줘 파일 보내줄게.

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

정n각형을 한 바퀴 굴릴 때 점 A의 이동 거리

동해가 길이가 1인 정n각형을 한 바퀴 굴릴 때 점 A가 지나가는 궤적의 지름 r을 통해 확인할 수 있습니다. 디자인은 깔끔하게, 원형은 반박할 수 있는 형태로 보여주세요. 무성합니다.

문제에서 구하는 것

변수: n (4각, 4.5각, 4.9999, 5.0001, 4.9999, 5.0001)

발상(직접, 정사각형, 정100각형에서 점 A의 이동 거리를 확인하고, 일반적인 정n각형으로 확장한 후 n의 치환 계의 값을 세고있다.)

목표 식

원 변의 길이 $= 1/n$

$$r_n = \sin(30/n) / \sin(30/n)$$

$$r(n) = (2n/n) \sin(30/n) - 2 \sin(30/n)$$

$$r(4.999999999) = 4/n = 1.277336$$

정3각형

점 A가 지나가는 궤적의 길이를 입력받아 보여주세요

조작

3각형, 4각형, 5.0001, 4.9999

$n = 3$

회전도 $= 2.00$

재생 속도 $= 1.00$

결과

점 A의 이동 거리: 1.396263

점 A의 이동 거리: 1.396263

점 A의 이동 거리: 2

(1) 동해가 길이가 1인 정n각형을 한 바퀴 굴릴 때 점 A가 지나가는 궤적의 지름 r을 통해 확인할 수 있습니다. 디자인은 깔끔하게, 원형은 반박할 수 있는 형태로 보여주세요. 무성합니다.

(2) 두 단계에서 회전도를 설정하고 n의 값을 입력받아 점 A의 이동 거리를 확인할 수 있습니다.

(3) n의 값을 정수나 소수로 입력하면 점 A의 이동 거리를 확인할 수 있습니다.

이동 거리는 1인 정n각형을 한 바퀴 굴릴 때 점 A가 지나가는 궤적의 길이를 입력받아 보여주세요.

10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

(1) 기본 도형 선택 및 비교

- 조작 방법: 화면 오른쪽 상단의 3각형, 4각형, 20각형, 100각형 버튼을 클릭한다.
- 기능: 둘레의 길이가 1인 정삼각형, 정사각형, 정20각형, 정100각형이 한 바퀴 굴러가는 모습을 즉시 확인할 수 있다. 특히 100각형의 경우 거의 원과 같은 형태로 나타나므로, 정 n 각형이 원에 가까워지는 과정을 직관적으로 관찰할 수 있다.

(2) 정 n 각형의 변의 수 직접 조절

- 조작 방법: n = 슬라이더를 좌우로 움직여 원하는 자연수 n 을 선택한다.
- 기능: 정 n 각형의 꼭짓점 개수가 실시간으로 변하며, 오른쪽 패널에 현재 선택한 n 값과 그에 대응하는 이동 거리 $d(n)$ 이 자동으로 계산되어 표시된다. 이를 통해 n 이 증가할수록 이동 거리가 일정한 값에 수렴하는 경향을 확인할 수 있다.

(3) 자동 재생 기능

- 조작 방법: ▶ 자동 재생 버튼을 클릭하면 애니메이션이 시작되고, 다시 클릭하면 일시 정지된다.
- 기능: 정 n 각형이 기준선 위를 한 바퀴 굴러가면서, 꼭짓점 A가 그리는 궤적이 주황색 곡선으로 나타난다. 각 꼭짓점에서 호가 이어지며 전체 이동 경로가 형성되는 과정을 시각적으로 확인할 수 있다.
- 다시 처음 상태로 돌아오기 위해서는 슬라이드 바를 다시 왼쪽으로 밀어 초기화해야 한다.)

(4) 재생 속도 조절

- 조작 방법: 재생 속도 슬라이더를 좌우로 움직인다.
- 기능: 애니메이션의 진행 속도를 자유롭게 조절할 수 있다. 속도를 낮추면 각 꼭짓점에서 회전하는 과정을 자세히 관찰할 수 있고, 속도를 높이면 한 바퀴 전체의 궤적을 빠르게 확인할 수 있다.

(5) 이동 거리 $d(n)$ 확인

- 조작 방법: n 값을 변경하거나 다른 도형 선택 버튼을 누른다.
- 기능: 오른쪽 패널의 $d(n)$ 영역에 현재 정 n 각형에 대해 계산된 점 A의 총 이동 거리가 소수 여섯째 자리까지 표시된다. 이를 이용하여 문제의 (1)의 삼각형과 사각형의 경우를 해결할 수 있다.

(6) 풀이 과정 확인

- 조작 방법: 오른쪽 하단의 풀이 요약 영역을 확인한다.
- 기능: 한 변의 길이가 $1/n$ 이라는 사실부터 시작하여, 호의 길이를 합해 $d(n)$ 을 구하고, 마지막으로 극한값에 도달하는 전체 논리를 간단한 식과 함께 정리하였다. 따라서 시뮬레이션 결과와 수학적 풀이를 동시에 이해할 수 있다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

[도움이 된 부분]

첫 번째로 문제 상황을 직관적으로 이해하는 데 큰 도움이 되었다. 처음에는 정 n 각형이 굴러갈 때 점 A의 자취가 어떤 형태로 나타나는지 상상하기 어려웠다. 하지만 시뮬레이터에서 도형이 실제로 회전하며 이동하는 모습을 관찰하자, 점 A의 자취가 단순한 곡선이 아니라 여러 개의 원호가 이어진 형태라는 사실을 한눈에 이해할 수 있었다. 특히 정삼각형, 정사각형, 정 n 각형 등을 비교해 보면서 n 이 증가할수록 자취가 점점 원의 사이클로이드에 수렴함을 쉽게 파악할 수 있었다.

두 번째로 풀이 과정의 핵심 아이디어를 얻는 데 도움이 되었다. 처음에는 이동 거리를 어떻게 일반화해야 할지 막막했지만, 시뮬레이터에서 각 단계의 회전을 관찰해 보니 모든 단계의 회전각은 일정하고 반지름만 달라진다는 구조를 발견할 수 있었다. 이를 통해 전체 이동 거리가 중심각 $\times$ 반지름의 반복 구조로 이루어진다는 사실을 직관적으로 이해할 수 있었고, 일반식을 세우는 방향을 잡는 데 도움이 되었다. 또한 n 을 크게 설정했을 때 자취가 거의 원처럼 보이는 것을 확인하며, 마지막 극한 계산이 적분과 연결된다는 점에 대한 시각적인 확신도 얻을 수 있었다.

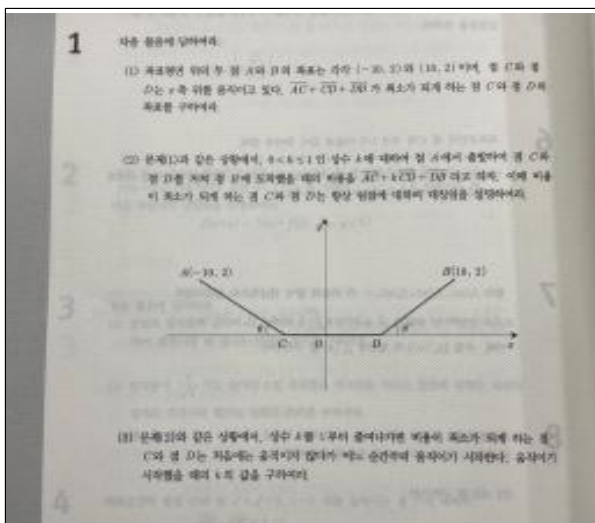
세 번째로 내가 직접 계산한 결과를 검토하는 데 유용했다. 예를 들어 정2 n 각형에서 반지름이 대칭 구조를 가진다는 점이나, n 이 커질수록 중심각이 점점 작아지는 현상을 실제 자취와 비교하며 확인할 수 있었다. 또한 내가 세운 일반식이 실제 움직임과 일치하는지도 시뮬레이터를 통해 검토할 수 있었기 때문에, 계산 과정의 오류를 줄이는 데

[도움이 되지 않은 부분 (한계점)]

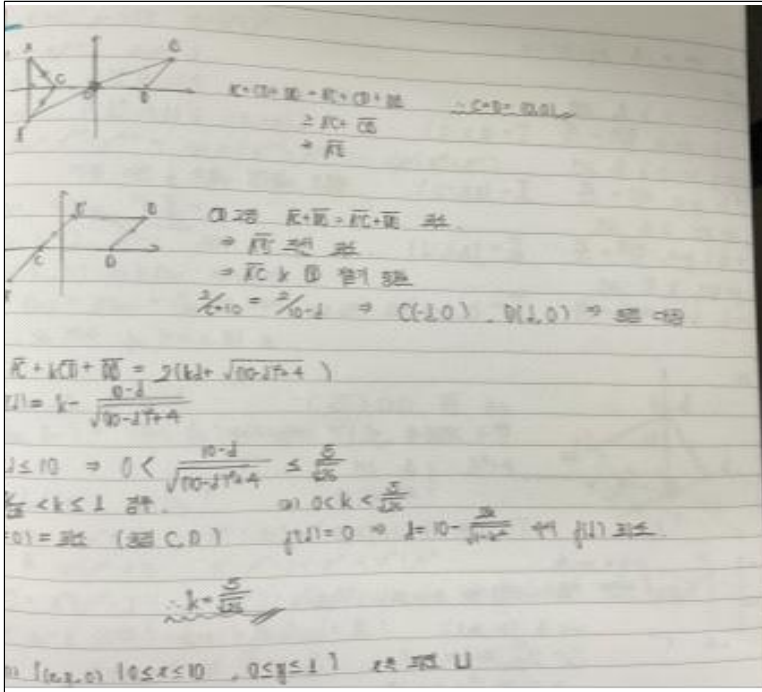
또한 시뮬레이터 내부 계산은 소수점 근사를 기반으로 이루어지기 때문에, 화면상으로는 자취가 완벽한 원처럼 보 이더라도 실제로는 미세한 오차가 존재할 수 있었다. 특히 n 이 매우 클 때 극한값에 가까워지는 모습은 확인할 수 있었지만, 왜 그런 극한값이 나오는지 엄밀하게 증명하기 위해서는 결국 적분과 극한 계산을 직접 수행해야 했 다. 따라서 시뮬레이터는 직관적인 아이디어를 얻고 계산 결과를 검토하는 데에는 매우 효과적이었지만, 엄밀한 수 학적 연역 과정을 대체하는 데에는 한계가 있었다.

이러한 한계를 보완하기 위해서는 시뮬레이터에서 관찰한 내용을 단순히 보는 것에서 끝내지 않고, 왜 그런 현상이 나타나는지를 직접 수식과 연결하는 과정이 필요하다고 생각한다. 예를 들어 사용자가 시뮬레이터에서 확인한 원호의 구조와 회전 과정을 바탕으로 직접 호의 길이 공식을 적용해 보고, 정n각형의 중심과 현의 성질을 이용해 일반 항을 유도하는 연습이 필요하다. 또한 극한 과정 역시 시각적인 변화만 관찰하는 것이 아니라, 실제로 적분과 극한 계산을 직접 전개하며 이해해야 사용자의 시뮬레이션에 대한 직관과 엄밀한 수학적 증명을 연결할 수 있다고 생각한다. 또는 사용자 친화적으로 증명 과정 또한 시뮬레이터에 추가하여 이해를 돕는 하나의 온라인 해설지를 만드는 것도 도움이 될 것 같다.

1. 내가 선택한 두 번째 문제 (캡처해서 붙여넣기 해도 됨)



2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)



3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

이 문제는 단순히 식을 계산하는 것에서 끝나는 것이 아니라, 매개변수 k의 값에 따라 점 C와 D의 최적 위치가 어떻게 달라지는지를 이해하는 것이 핵심이다. 식 $AC + kCD + DB$ 를 최소로 만드는 과정에서 $k = \frac{5}{\sqrt{26}}$ 을 기준으로 최솟값이 경계점 $d=0$ 에서 결정되기도 하고, 내부의 특정 위치에서 결정되기도 한다. 이러한 변화는 식만 보고 파악하기보다 직접 점 C와 D가 움직이는 모습을 관찰할 때 훨씬 직관적으로 이해할 수 있다. 또한 함수 $f(d)$ 와 도함수 $f'(d)$ 의 그래프를 함께 보면 최솟값이 형성되는 원리와 임계값의 의미를 시각적으로 확인할 수 있다. 따라서 이 문제는 계산 결과뿐 아니라 최적화 구조와 매개변수 변화에 따른 현상을 탐구하는 것이 중요하므로, 시뮬레이션을 통해 접근하는 것이 가장 효과적이라고 판단하였다.

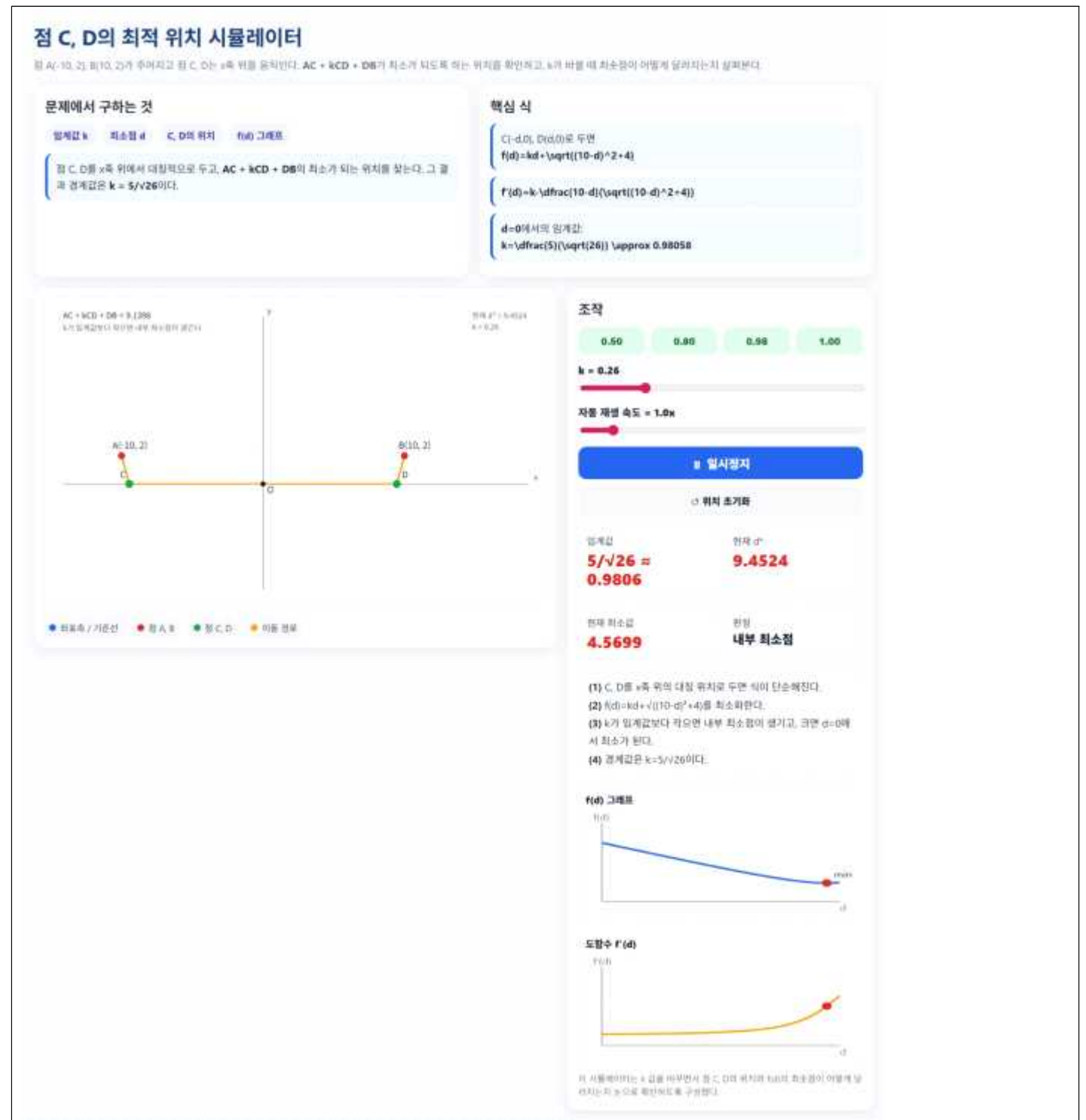
4. 사용한 AI 플랫폼을 모두 적으세요.

ChatGPT

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

이제 2번째 문제를 시작할거야
이 문제의 1,2번은 대칭성을 증명하는 문제니깐 이걸 생략하고 3번 문제에서 최솟값을 가지는 위치의 변화를 시각적으로 보여주는 것을 초점으로 시뮬레이션을 만들거야.
3번에서 구한 함수의 변화를 초점으로 해줘,
동일하게 너가 직접 문제를 풀어보고 답이랑 비교하고 시뮬레이션을 만들면 된다.
한번 해봤으니깐 잘할 수 있을 것이라 믿음.

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

첫 번째 시뮬레이션의 학습으로 틀과 디자인은 이전과 비슷하게 만족스러웠지만, 화면이 너무 세로로 길었고, 그래프의 위치가 한눈에 들어오지 않았다. 또한 수식이 올바르게 표현되지 않고 영문을 수정해야 했다. 최솟값이 변하는 순간 또한 f' 그래프를 표현함으로써 시각화를 잘할 수 있었다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

(2) ㅇㅋ 일단 잘했는데 그래프가 왼쪽에 모여 있으면 좋겠어. 너무 세로로 긴 느낌이라서 그리고 d*가 무엇인지 모르겠고, 수식이 기호가 아니 frac이나 sqrt처럼 영어로 표시되고 있어 고쳐줘.

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

점 C, D의 최적 위치 시뮬레이터

점 A(-10, 2), B(10, 2)가 주어지고 점 C, D는 x축 위의 움직인다. $AC + kCD + DB$ 가 최소가 되도록 하는 위치를 확인하고, k에 따라 최솟값이 어떻게 달라지는지 살펴본다.

문제에서 구하는 것

임계값 k 최소점 d C, D의 위치 f(d) 그래프

점 C, D를 x축 위에서 대칭적으로 두고, $AC + kCD + DB$ 의 최소가 되는 위치를 찾는다. 그 결과 경계값은 $k = 5/\sqrt{26}$ 이다. 이 값이 기준이 되어 최솟값이 d = 0에서 내부로 이동할지 결정된다.

핵심 식

C(-d, 0), D(d, 0)로 두면
 $f(d) = kd + \sqrt{((10-d)^2 + 4)}$

$F(d) = k - (10-d) / \sqrt{((10-d)^2 + 4)}$

d=0에서의 임계값:
 $k = 5/\sqrt{26} \approx 0.98058$

$AC + kCD + DB = 7.0068$
 k가 임계값보다 작으면 내부 최솟값이 생긴다.

현재 최적 d = 9.6910
 k = 0.15



● 좌표축 / 기준선 ● 점 A, B ● 점 C, D ● 이동 경로

조작

0.50

0.80

0.98

1.00

k = 0.15

자동 재생 속도 = 1.0x

일시정지

위치 초기화

임계값

$5/\sqrt{26} \approx$
 0.9806

현재 최적 d

9.6910

현재 최솟값

3.5034

판정

내부 최솟점

f(d) 그래프



도함수 f'(d)



- (1) C, D를 x축 위의 대칭 위치로 두면 식이 단순해진다.
- (2) $f(d) = kd + \sqrt{((10-d)^2 + 4)}$ 를 최소화한다.
- (3) k가 임계값보다 작으면 내부 최솟값이 생기고, 크면 d=0에서 최소가 된다.
- (4) 경계값은 $k = 5/\sqrt{26}$ 이다.

이 시뮬레이터는 k 값을 바꾸면서 점 C, D의 위치와 f(d)의 최솟값이 어떻게 달라지는지 눈으로 확인하도록 구성했다.

10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

(1) k값 직접 조절

- 조작 방법: 화면 왼쪽의 k 슬라이더를 좌우로 움직인다.
- 기능: k 값이 실시간으로 변하면서 점 C, D의 위치와 그래프의 형태가 함께 변화한다. 이를 통해 k값에 따라 함수의 최소가 나타나는 위치가 어떻게 달라지는지를 직관적으로 확인할 수 있다.

(2) 예시 값 빠른 선택 기능

- 조작 방법: 슬라이더 아래의 예시 버튼을 클릭한다.
- 기능: 특정한 대표 k 값들이 즉시 적용되며, 그래프와 도형이 자동으로 갱신된다. 이를 이용해 임젯값보다 작은 경우, 같은 경우, 큰 경우를 빠르게 비교할 수 있다.

(3) 함수 그래프 확인

- 조작 방법: 화면 왼쪽의 그래프 영역을 확인한다.
- 기능: 현재 k 값에 대한 함수의 전체 형태와 최솟값의 위치를 시각적으로 볼 수 있다. 그래프의 기울기 변화와 최솟값의 이동을 함께 관찰할 수 있으므로, 함수가 어떤 조건에서 최소가 되는지 직관적으로 이해할 수 있다.

(4) 핵심 식 및 계산 결과 확인

- 조작 방법: 화면 오른쪽의 핵심 식 영역을 확인한다.
- 기능: 현재 문제 풀이에 사용되는 핵심 관계식과 도함수 정보, 그리고 임계값이 정리되어 표시된다. 따라서 그래프에서 관찰한 현상을 수식과 연결하여 이해할 수 있으며, 실제 풀이 과정과 시뮬레이션 결과를 동시에 비교할 수 있다.

(5) 최솟값 위치 분석

- 조작 방법: k 값을 조금씩 변화시키며 그래프의 최솟값 위치를 비교한다.
- 기능: k 값이 변할 때 최솟값이 내부에 존재하는지, 경계에서 나타나는지가 달라지는 과정을 실시간으로 관찰할 수 있다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

[도움이 된 부분]

첫 번째로 문제 상황을 빠르게 이해하는 데 도움이 되었다. 처음에는 k 값이 달라질 때 점 C와 D의 최적 위치가 왜 달라지는지 감이 잘 오지 않았는데, 시뮬레이터에서 슬라이더를 움직이며 그래프와 도형의 변화를 함께 보니 함수의 모양이 어떻게 바뀌는지 직관적으로 파악할 수 있었다. 특히 k가 작을 때는 내부에 최솟값이 생기고, k가 커질수록 경계 쪽으로 이동하는 흐름을 눈으로 확인할 수 있어서 문제의 핵심을 이해하는 데 큰 도움이 되었다.

두 번째로 풀이 과정의 방향을 잡는 데 유용했다. 단순히 식만 보면 최솟값이 어디서 생기는지 판단하기 어려웠지만, 시뮬레이터의 그래프를 통해 함수의 기울기 변화와 최솟값의 이동을 반복해서 관찰할 수 있었다. 그 결과 임젯값을 기준으로 최솟값의 위치가 바뀐다는 점을 자연스럽게 받아들일 수 있었고, 이후 계산 과정도 더 명확하게 정리할 수 있었다.

마지막으로 내가 구한 답을 검토하는 데 도움이 되었다. 직접 계산한 임젯값이 실제 시뮬레이터에서 경계가 바뀌는 지점과 일치하는지를 확인하면서, 내 풀이가 맞는지 점검할 수 있었다. 특히 몇 가지 대표 k 값을 넣어 보았을 때 그래프의 최솟값이 내가 예상한 위치와 같게 나타나서 계산 결과에 대한 확신을 얻을 수 있었다.

[도움이 되지 않은 부분]

다만 시뮬레이터는 결과를 보여줄 뿐, 왜 그런 결과가 나오는지까지 엄밀하게 증명해 주지는 못했다. 예를 들어 임젯값이 정확히 왜 그 값이 되는지, 그리고 그 값을 기준으로 최솟점이 왜 이동하는지는 그래프를 보는 것만으로는 알 수 없었다. 결국 실제 서술형 답안에서는 함수식을 정리하고, 조건에 따라 경우를 나누어 직접 논리적으로 증명해야 했다. 또한 시뮬레이터는 내부적으로 근삿값을 사용하므로, 화면에서 보이는 최솟값이 정확한 수학적 해와 완전히 같다고 단정할 수는 없었다. 그래서 시각적으로는 충분히 이해되더라도, 최종 답안에서는 반드시 대수적인 계산과 검산이 필요했다. 즉, 이 시뮬레이터는 문제를 이해하고 검토하는 데에는 효과적이었지만, 엄밀한 증명 자체를 대신할 수는 없었다.

[한계점에 대한 대안]

이런 한계를 보완하기 위해서는 시뮬레이터에서 관찰한 변화와 그래프의 모양을 바탕으로, 실제로 함수의 식을 정리하고 도함수의 부호 변화까지 직접 확인하는 과정이 필요하다. 즉, 시각적인 직관은 시뮬레이터로 얻고, 그 직관을 수학적으로 완성하는 것은 직접 풀이를 통해 마무리해야 한다.

[마무리]

1. 두 문제에 대한 시뮬레이터를 기획하고 활용하여 문제를 해결하는 전 과정에서, 생성형 AI가 제공할 수 있는 가치와 반드시 인간(학생 본인)이 주도해야 하는 수학적 사고의 영역을 구분하여 서술하세요.

두 문제를 해결하는 과정에서 생성형 AI는 시뮬레이터 제작과 아이디어 정리에 매우 유용한 도구가 될 수 있었지만, 문제의 핵심 원리를 이해하고 이를 수학적으로 증명하는 과정까지 대신할 수는 없었다. 실제로 문제 해결 과정에서는 AI가 효과적으로 도와줄 수 있는 영역과 반드시 학생이 직접 수행해야 하는 수학적 사고의 영역이 분명하게 구분된다고 느꼈다.

먼저 생성형 AI가 가장 큰 가치를 제공한 부분은 시뮬레이터의 구현 과정이었다. 두 문제 모두 단순 계산 문제가 아니라 점의 위치나 도형의 형태가 계속 변화하는 동적 상황을 다루고 있었기 때문에, 이를 직접 코드로 구현하려면 상당한 시간이 필요했다. 이때 AI는 HTML, CSS, JavaScript 기반의 인터랙티브 시뮬레이터 구조를 빠르게 생성해 주었고, 슬라이더, 자동 재생, 그래프, 실시간 계산 기능 같은 시각적 요소를 구현하는 데 도움을 주었다. 또한 사용자가 어떤 버튼을 누르면 어떤 정보가 표시되어야 하는지와 같은 UI 구성 아이디어를 제안해 주었기 때문에, 직관적인 시뮬레이터를 제작할 수 있었다. 프로그래밍을 전공한 사람이 아니더라도 이제는 생성형 AI를 다룰 수만 있으면, 과거에는 매우 오랜 시간에 걸쳐 제작해야 하는 프로그램을 단시간에 제작할 수 있어 우리가 스스로 할 수 있는 학습의 범주가 늘어난 느낌이었다. 또한 AI는 탐구 과정에서 떠오른 아이디어를 정리하거나, 복잡한 내용을 문장 형태로 정리하는 데에도 도움이 되었다. 예를 들어 시뮬레이터의 사용 방법, 도움이 된 점과 한계점, 문제 해결 과정 등을 보고서 형식으로 구조화하는 과정에서는 AI가 초안을 제시해 주었고, 이를 바탕으로 표현을 수정하며 내용을 정리할 수 있었다.

하지만 문제의 핵심적인 수학적 사고 과정은 반드시 학생인 내가 직접 수행해야 했다. 예를 들어 첫 번째 문제에서는 점의 자취가 왜 특정한 곡선이 되는지, 어떤 대칭성이 존재하는지, 넓이를 구하기 위해 왜 특정 구간만 적분하면 되는지를 스스로 분석해야 했다. 정 n 각형의 자취가 왜 여러 개의 원호로 구성되는지, 반지름과 회전각 사이에 어떤 관계가 있는지, 마지막 극한 과정이 왜 적분과 연결되는지를 직접 이해하는 과정이 필수적으로 요구되었다. 시뮬레이터는 결과를 보여줄 수는 있었지만, 그 결과가 왜 성립하는지를 논리적으로 설명해 주지는 못했다. 특히 수학적 증명 과정은 AI가 대신할 수 없는 영역이라고 느꼈다. 실제 서술형 풀이에서는 삼각함수, 기하적 성질, 극한과 적분 등의 개념을 이용해 식을 유도하고 논리를 전개해야 했는데, 이러한 과정은 단순히 결과를 받아 적는 것으로 해결되지 않았다. 만약 시뮬레이터에서 나타나는 현상을 스스로 해석하지 못한다면, AI가 제시한 식이나 결과 역시 제대로 이해할 수 없다는 점을 느꼈다. 즉, AI는 직관과 도구를 제공할 수는 있지만, 수학적 의미를 해석하고 논리로 연결하는 역할은 결국 인간의 사고가 담당해야 했다. 평소에도 문제를 풀면서 모르거나 풀이 과정이 이해되지 않는 부분을 AI에게 질문하지만, 결코 AI는 우리 학습의 보조도구만으로 사용되어야 하고 우리가 직접 사고하고 이해하는 과정이 필수적이다. 즉, 공부와 학습은 최종적으로 우리 뇌와 사고에 들어가야 하기 때문에 모든 것을 AI가 담당할 수는 없다고 생각했다.

2. 복잡한 수학 문제를 AI와 협업하여 시뮬레이터로 구현해 본 경험이, 향후 자신이 희망하는 진로 분야에서 실무적인 문제를 해결하는 데 어떤 시사점을 주는지 서술하세요.

이번 탐구를 통해 복잡한 수학 문제를 단순히 계산으로만 해결하는 것이 아니라, 변화 과정을 직접 시각화하고 상호작용이 가능한 형태로 구현해 보는 경험은 앞으로 희망하는 산업공학 및 디자인 분야와 매우 밀접하게 연결된다고 느꼈다. 특히 복잡한 현상을 사용자가 직관적으로 이해할 수 있도록 구조화하고 표현하는 과정 자체가 실제 문제 해결 방식과 유사하다는 점이 인상적이었다. 산업공학 분야에서는 생산 공정, 물류 시스템, 최적화 문제처럼 여러 요소가 동시에 영향을 주고받는 복잡한 상황을 다루게 된다. 이런 문제들은 단순히 수식만으로 분석하기보다, 변화 과정을 시각적으로 확인하고 전체 흐름을 이해하는 과정이 중요하다고 알고 있다. 이번 탐구에서도 도형의 움직임과 점의 자취를 직접 관찰하면서, 계산만으로는 쉽게 발견하지 못했던 규칙성과 구조를 직관적으로 이해할 수 있었다. 이를 통해 실제 문제 해결에서도 데이터를 단순히 계산하는 것을 넘어, 변화 과정을 시각화하고 구조를 분석하는 능력이 중요하다는 점을 느낄 수 있었다. 또한 디자인 분야와의 연결성도 크게 느껴졌다. 시뮬레이터를 제작할 때는 단순히 기능을 구현하는 것만이 아니라, 사용자가 어떤 순서로 정보를 받아들이는지, 어떤 배치가 핵심 내용을 가장 잘 드러내는지까지 함께 고민해야 했다. 예를 들어 슬라이더를 통해 값을 직접 변화시켜 보게 하거나, 그래프와 핵심 정보를 동시에 배치한 것은 모두 사용자가 문제의 구조를 더 쉽게 이해할 수 있도록 하기 위한 과정이었다. 또한 두 번째 문제의 핵심인 그래프들을 처음에는 생성형 AI가 글과 함께 떨어진 곳에 배치하였지만, 그래프를 한 곳에 옮겨 도함수와 실제 그래프의 변화를 함께 보니, 최솟값의 변화를 보다 직관적으로 관찰할 수 있었다. 이를 통해 디자인은 단순히 시각적으로 아름다운 결과물을 만드는 것이 아니라, 복잡한 정보를 효과적으로 전달하고 사용자의 이해를 돕는 과정이라는 점을 체감할 수 있었다. 하나의 결과를 얻는 것에서 끝나는 것이 아니라, 다른 사람이 그 과정을 쉽게 이해하고 활용할 수 있도록 만드는 과정 역시 중요한 역량이라는 점을 배울 수 있었다.

3. 보고서를 작성하며 느낀점을 간단히 작성하세요.

이번 보고서를 작성하면서 단순히 문제의 정답을 구하는 것보다, 문제의 구조를 이해하고 그것을 다른 사람이 직관적으로 이해할 수 있도록 표현하는 과정이 훨씬 중요하다는 점을 느꼈다. 평소에는 문제를 풀 때 식을 세우고 계산하는 데에만 집중했지만, 실제로 시뮬레이터를 제작해 보니 도형의 움직임이나 함수의 변화 과정을 직접 눈으로 확인할 수 있었고, 그 과정에서 계산만으로는 쉽게 발견하지 못했던 규칙성과 대칭성을 더 직관적으로 이해할 수 있었다. 특히 정 n 각형의 자취가 여러 개의 원호로 이루어진다는 점이나, n 이 커질수록 자취가 원의 사이클로이드처럼 변해 간다는 점은 시뮬레이션을 통해 훨씬 명확하게 느낄 수 있었다. 또한 단순히 결과를 얻는 것에서 끝나는 것이 아니라, 다른 사람이 이 문제를 쉽게 이해할 수 있도록 시뮬레이터의 구성과 설명 방식을 고민하는 과정도 기억에 남았다. 어떤 정보를 먼저 보여줘야 직관적으로 이해할 수 있는지, 어떤 버튼이나 슬라이더가 있어야 사용자가 직접 탐구할 수 있는지를 계속 수정하면서, 복잡한 내용을 효과적으로 전달하는 과정의 중요성을 느꼈다. 이를 통해 수학은 단순한 계산 과목이 아니라, 복잡한 현상을 논리적으로 분석하고 구조화해야 한다는 점을 다시 생각하게 되었다. 특히 하나의 결과를 얻는 것보다 왜 그런 결과가 나오는지 직관적으로 이해하고 설명할 수 있어야 진짜로 문제를 이해한 것이라는 점을 느낄 수 있었고, 앞으로도 수학 문제를 단순 계산으로만 접근하지 않고 시뮬레이션 이외의 다양한 방식으로 탐구해 보고 싶다는 생각이 들었다.